OBJECT ORIENTED PROGRAMMING
CCS20704, DCS20504, CCS23803

Received Date : 3 September 2026
Submission Due Date: 23 October 2026
Lecturer : Dr. Jamal Abdullahi
Weightage : 15%
Semester : SEPTEMBER 2026

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:		
I declare that: • ***This assignment is my/our own work.*** • ***I/we understand what is meant by plagiarism.*** • ***My lecturer has the right to deduct my marks in case of:*** - ***Late submission*** - ***Any plagiarism found in my assignment.***		
NAME: THAVAANITHAN SUNDRALINGAM **MATRICS NO: 012026021321** 	**NAME: VHETHAASHINI A/P THANGARAJU** **MATRICS NO: 012025092401** 	**NAME: PRIYA LOSHNI A/P SELVAKUMAR** **MATRICS NO: 012026070337** 

1.0 Introduction

Object-oriented programming (OOP) is a programming approach that organises a program using classes and objects. A class defines the attributes that describe an object and the methods that operate on its data. OOP helps organise code into manageable sections and supports code reuse, data protection and easier maintenance.

This assignment presents a Car Management System developed using Java. The system allows a user to enter car information, view its details, update mileage, estimate resale value, check a service threshold and change the car's availability.

A car was selected because it has clear properties and practical operations that are suitable for demonstrating OOP concepts. The project focuses on classes, objects, constructors, encapsulation, methods, selection structures and looping structures. The system is a console-based educational application rather than a complete commercial vehicle management system.

2.0 Objectives

The objectives of this project are:

1. To design a Java class representing a car with seven distinct attributes.
2. To apply encapsulation by declaring attributes as private and providing getters and setters.
3. To implement default and parameterized constructors for object initialisation.
4. To develop meaningful methods for displaying information, estimating resale value and checking service requirements.
5. To demonstrate user input, selection, looping and attribute manipulation in the main class.
6. To compile, execute and test the system using sample car information.

3.0 Class and Object Design

The system consists of two main classes: Car and CarApp.

The Car class acts as a blueprint for a car object. It stores the car's attributes and provides methods to read, update and process the information. The CarApp class contains the main() method, accepts user input and allows the user to interact with the car object through a menu.

During testing, an object named car1 was created using the parameterized constructor. The object represented a Proton X50 manufactured in 2020.

3.1 Attributes

Attribute	Data type	Purpose
brand	String	Stores the car brand
model	String	Stores the car model
year	int	Stores the manufacturing year
price	double	Stores the car price in RM
mileage	int	Stores mileage in kilometres
fuelType	String	Stores the fuel type
available	boolean	Indicates whether the car is available

The attributes use different data types according to the information being stored. String stores text, int stores whole numbers, double supports prices with decimal values, and boolean represents a true-or-false status.

3.2 Class Relationship Diagram

The following simplified UML-style class diagram shows the relationship between the main class and the car class.

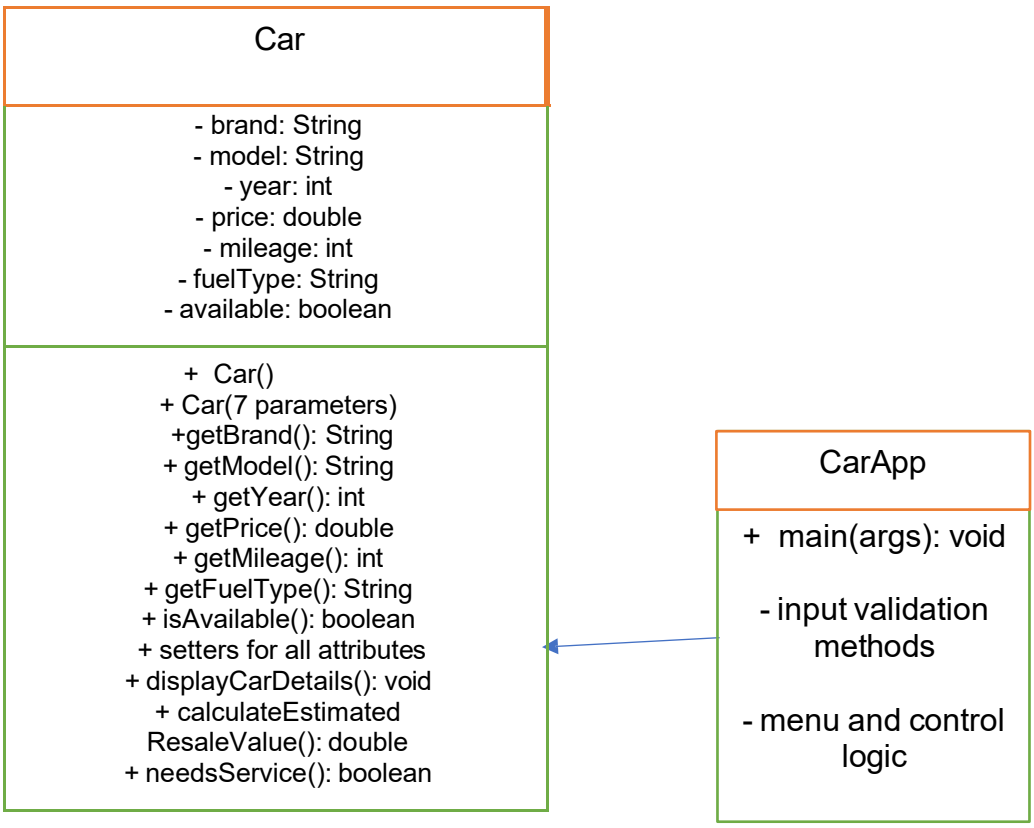


Figure 3.1: Simplified class design of the Car Management System.

The minus sign (-) represents private members, while the plus sign (+) represents public members. CarApp creates and interacts with a Car object through its public methods instead of accessing the car's private attributes directly.

4.0 Constructors and Encapsulation

4.1 Constructors

The Car class contains two constructors.

The default constructor, Car(), initialises the attributes with placeholder values such as "Not set", zero values and false for availability. This allows an object to be created before actual car information is supplied.

The parameterized constructor accepts values for all seven attributes. It calls this() to initialise the default values first, then uses the setters to assign the supplied information. This approach avoids repeating initialisation logic and allows the setter validation rules to be applied.
In the current program, CarApp uses the parameterized constructor because the user enters the car details before the menu appears.

4.2 Encapsulation

Encapsulation combines data and related operations within a class while restricting direct access to internal data. In this project, all seven attributes in Car are declared private. This prevents other classes from directly changing the fields.

Public getters provide controlled read access, while setters provide a way to request changes to the attributes. For example, getMileage() returns the current mileage and setMileage() updates it when the supplied value is non-negative.

The setters also apply basic validation. Brand, model and fuel type reject null or blank text. The manufacturing year must be between 1886 and 2026, price must be finite and non-negative, and mileage must be non-negative. The availability setter accepts a Boolean value. These rules help maintain reasonable data values. However, the current program does not provide a database or permanent storage, so the information is retained only while the application is running.

5.0 Methods and Their Purposes

The Car class includes getters and setters for all seven attributes, together with three additional methods.

Method	Purpose
getBrand()	Returns the brand
getModel()	Returns the model
getYear()	Returns the manufacturing year
getPrice()	Returns the price
getMileage()	Returns the current mileage
getFuelType()	Returns the fuel type
isAvailable()	Returns the availability status
Seven setter methods	Update the attributes through controlled access
displayCarDetails()	Displays all seven car attributes
calculateEstimatedResaleValue()	Calculates a simplified resale estimate
needsService()	Checks whether mileage has reached 10,000 km

The displayCarDetails() method presents the stored information in a readable format. The calculateEstimatedResaleValue() method uses the manufacturing year and original price to estimate the current value. The needsService() method returns true when mileage is at least 10,000 km and false otherwise.The resale calculation uses a simplified depreciation rate of 10% per year, capped at 70%. This rule is intended for demonstrating calculations in OOP and should not be treated as a professional vehicle valuation. Similarly, the 10,000 km service threshold is an example used for this assignment, not a universal vehicle maintenance recommendation.

6. Main Class and Control Structures

The CarApp class contains the main() method and controls the interaction between the user and the Car object. It uses Java's Scanner class to receive input through the console.

6.1 Selection structures

The switch statement processes the user's menu choice. Each case performs a different operation, such as displaying car details, updating mileage, calculating the estimated resale value, checking service requirements, changing availability or exiting the program.

The if-else structure is used to determine whether the mileage has reached the example service threshold. It is also used when checking whether a mileage update is non-negative and when displaying the availability status.

6.2 Looping structures

A do-while loop repeatedly displays the menu until the user selects option 6. This allows several operations to be performed on the same object during one program session.

The updated main class also uses while loops to validate selected inputs, including the availability response and menu selection. These loops repeat the relevant prompt when an invalid response is entered.

6.3 Getter and setter interaction

The main class demonstrates controlled object interaction through public methods. For example, it uses setMileage() to update the mileage and getMileage() to display the new value. The availability feature uses isAvailable() to read the current status and setAvailable() to change it.

This approach demonstrates encapsulation because the main class does not directly modify the private attributes of Car.

7. Testing and Results

The program was compiled and executed successfully using PowerShell in the project directory. The following tests were performed using a Proton X50 as the sample vehicle.

7.1 Initial test data

Input field	Test value
Brand	PROTON
Model	X50
Manufacturing year	2020
Price	RM80,000
Initial mileage	40,000 km
Fuel type	PETROL
Availability	YES

7.2 Test results

Test	Action performed	Actual result
1	Compiled Car.java and CarApp.java	Compilation completed without errors
2	Selected option 1	All seven car attributes were displayed
3	Selected option 2 and entered 56,000	Mileage updated to 56,000 km
4	Selected option 3	Estimated resale value displayed as RM32,000.00
5	Selected option 4	Service check was recommended
6	Selected option 5	Availability changed from Yes to No
7	Selected option 1 again	Displayed the updated mileage and availability
8	Updated mileage to 78,000 and then 109,000	Both mileage updates succeeded
9	Selected option 5 again	Availability changed from No to Yes
10	Selected option 6	The program displayed "Program closed." and ended

The test results show that the main menu and the implemented operations functioned as expected for the supplied test values. The program also demonstrated that updates made through setters were reflected when the object details were displayed again.

7.3 Resale value calculation

For the test vehicle, the original price was RM80,000 and the manufacturing year was 2020. Using 2026 as the calculation year, the car's age was six years.

The estimated depreciation rate was:
6 years × 10% = 60%

The estimated resale value was:
RM80,000 × (1 − 0.60) = RM32,000

This matches the program output of RM32,000.00. Updating mileage did not change this estimate because the current calculation uses the manufacturing year and original price, rather than mileage.

7.4 Screenshot evidence

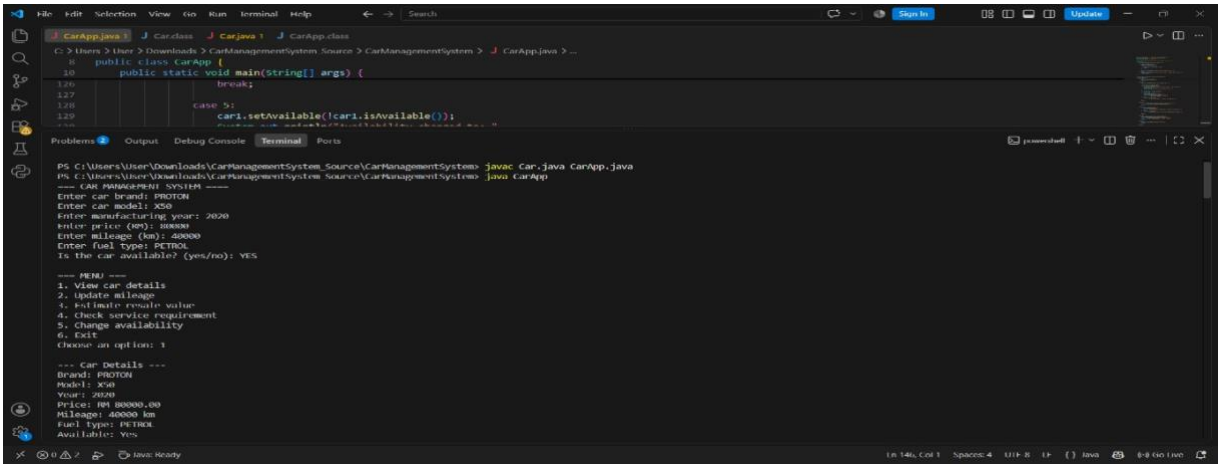


Figure 7.1: Initial car details. Show the console displaying the Proton X50 information after selecting option 1



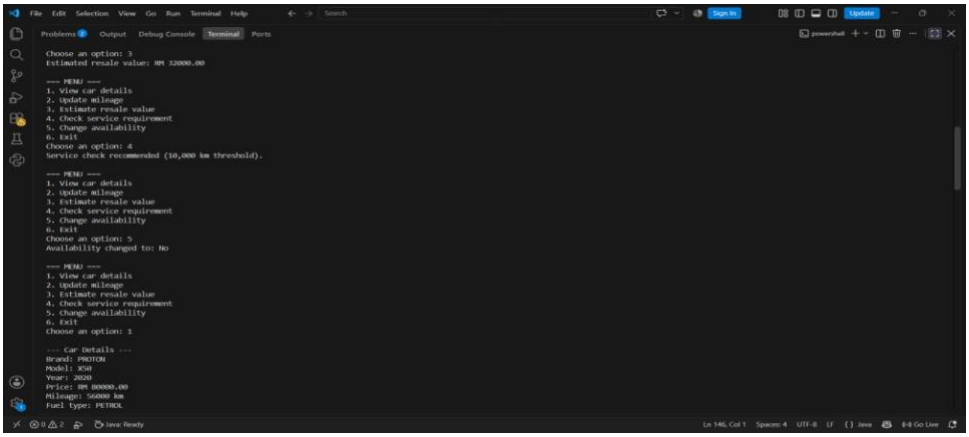








Figure 7.2 : Functional operations. Show the mileage update, resale estimate, service recommendation and availability change. If necessary, use two screenshots to keep the text readable.

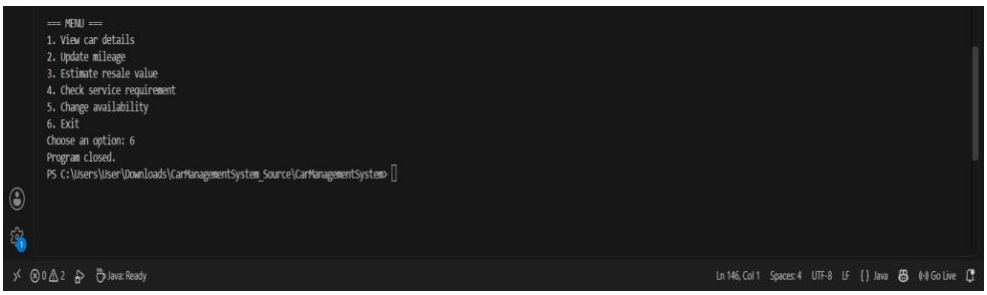


Figure 7.3 : Successful program termination. Show the final Program closed. message.

8.0 Discussion and Limitations

The project demonstrates the basic application of OOP through a practical car-related example. The use of private attributes and public getters and setters provides controlled access to data. Constructors make it possible to initialise objects in different ways, while the additional methods perform operations based on the car's attributes. The menu structure makes the program easy to use for basic operations. Testing also confirmed that the same object could be updated and viewed repeatedly without restarting the application. However, the current system has several limitations. It manages only one car object during a program session, does not save data permanently, and uses simplified rules for resale estimation and service checking. The resale calculation does not consider actual market prices, car condition or mileage. Future improvements could include storing multiple cars, saving records to a file or database, and using more realistic valuation and maintenance information.

9.0 Conclusion

In conclusion, the Car Management System demonstrates how Java can be used to apply the main concepts of object-oriented programming. The Car class organises seven attributes and related methods, while private fields and getters and setters support encapsulation. The two constructors provide different ways to initialise a car object.The CarApp class demonstrates user input, selection, looping and interaction with an object. The completed test run showed that the program could display car details, update mileage, calculate the simplified resale value, check the service threshold, change availability and exit successfully. Overall, this assignment strengthened the understanding of classes, objects, constructors, encapsulation and control structures through a practical programming task. Further development could improve data storage, validation and the realism of the calculations.

Appendix A: Compilation and Execution

The following commands were used to compile and run the Java application from its project directory:

```
javac Car.java CarApp.java
java CarApp
```

The source files Car.java and CarApp.java are submitted in GitHub project directory so that the application can be compiled and executed.

https://github.com/nithannn/OOP-Assignment-1-Car-Management-System/tree/main